



ICG:

Animação e Interação

Prof^a. Tainá Isabela



Introdução à Animação e Detecção de Colisão em Computação Gráfica

O estudo da computação gráfica moderna ultrapassa a mera renderização de imagens estáticas, exigindo a inserção de dinamismo e interatividade para a construção de ambientes virtuais imersivos.

Através do uso da API OpenGL, serão discutidos os modelos matemáticos e físicos necessários para simular fenômenos visuais complexos em tempo real.

Compreender esses mecanismos é fundamental para o desenvolvimento de simulações científicas, jogos eletrônicos e interfaces gráficas interativas de alta performance.

Evolução Histórica e os Pilares da Animação

A animação contemporânea se fundamenta na intersecção entre a arte tradicional, a computação gráfica e as leis da física. Historicamente, o conceito de dar vida a personagens e cenários evoluiu de métodos puramente manuais e bidimensionais para algoritmos complexos de renderização e simulação em tempo real.

Assim, o movimento computacional divide-se entre abordagens ativas, guiadas pelo controle direto do desenvolvedor, e abordagens passivas, governadas por modelos computacionais e físicos.

Os Princípios Clássicos da Animação e a Ilusão de Vida

Em 1981, os animadores Frank Thomas e Ollie Johnston sistematizaram doze princípios fundamentais no livro "The Illusion of Life", baseados nas décadas de aprendizado dos estúdios Disney.

Esses princípios, que isolaram procedimentos técnicos de movimentação e os transformaram em regras universais, continuam válidos no desenvolvimento de gráficos tridimensionais digitais.

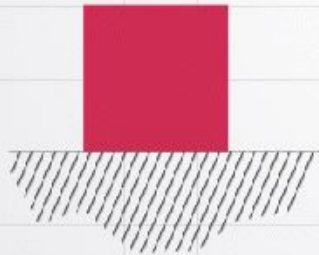
Conceitos como a deformação física dos corpos, a preparação do espectador para o movimento iminente e o acompanhamento inercial de ações secundárias são traduzidos matematicamente no código do pipeline gráfico. O domínio dessa terminologia e de sua aplicação prática diferencia simulações mecânicas rígidas de animações organicamente fluidas e naturais.

Princípios de Deformação e Inércia: Squash and Stretch

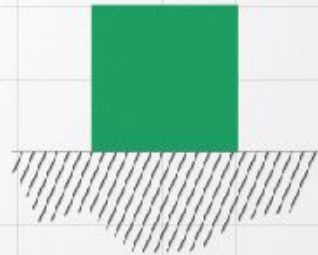
O princípio de espremer e esticar é utilizado para transmitir a sensação de rigidez, flexibilidade e peso de um objeto em movimento. Quando uma esfera colide contra o solo ou sofre uma aceleração abrupta, sua geometria deve se deformar temporariamente para refletir a atuação das forças físicas antes de retornar ao estado normal.

Em termos computacionais utilizando OpenGL, esse efeito é obtido aplicando matrizes de escala não uniformes sobre os vértices do modelo no Vertex Shader. É imperativo que o volume total do objeto permaneça constante durante a deformação; se a altura for reduzida em uma proporção matemática, a largura e a profundidade devem ser expandidas correspondentemente.

SEM
SQUASH AND STRETCH



COM
SQUASH AND STRETCH



Preparação Espacial e Temporal: Antecipação e Arcos

A antecipação consiste em preparar o espectador para uma ação principal por meio de um movimento menor na direção oposta, como um personagem que se agacha antes de saltar. Paralelamente, o princípio dos arcos dita que a maioria dos movimentos biológicos e mecânicos segue trajetórias curvas e parabólicas em vez de linhas retas estritas.

No desenvolvimento gráfico, trajetórias retilíneas criam uma percepção de artificialidade robótica indesejada. Programar arcos de movimento em OpenGL requer a interpolação de posições ao longo de funções polinomiais ou curvas de Bézier, calculando coordenadas tridimensionais dinâmicas baseadas no tempo.

SEM
ANTICIPATION



COM
ANTICIPATION



**SEM
ARCOS**



**COM
ARCOS**



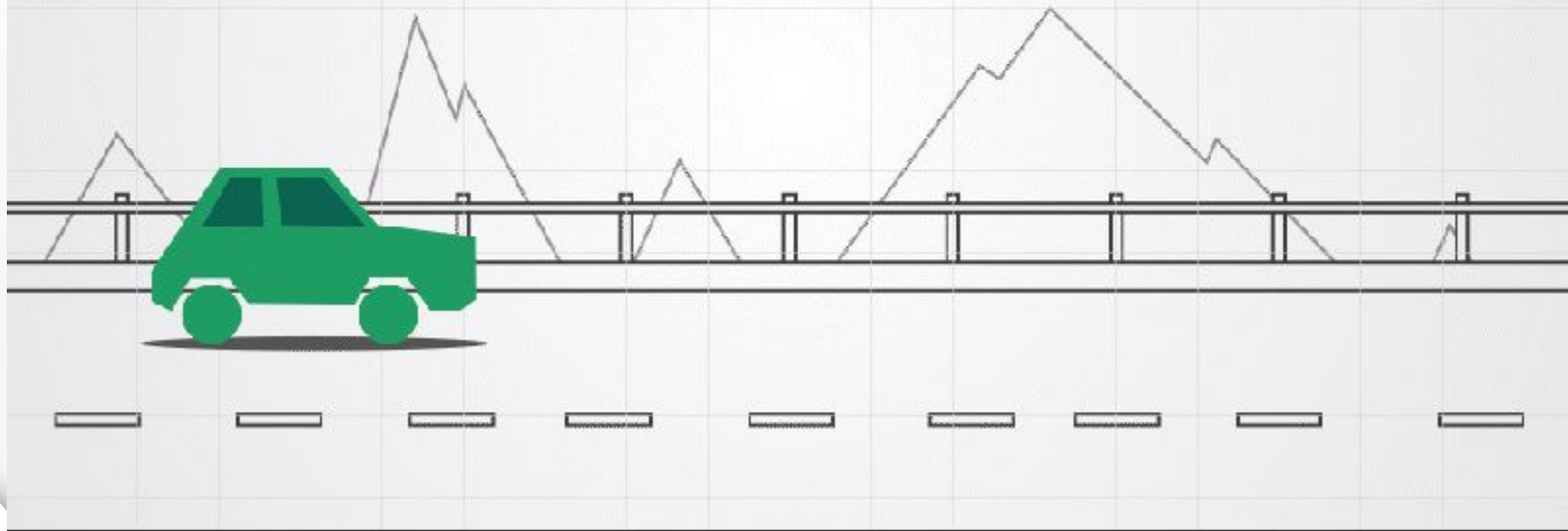
Dinâmica de Velocidade: Slow In and Slow Out

O princípio do amortecimento, estabelece que os objetos necessitam de tempo para acelerar e desacelerar devido à inércia natural do mundo físico. Uma animação linear simples, onde a velocidade permanece idêntica do início ao fim, gera movimentos abruptos e mecânicos na tela.

Para implementar a suavização de velocidade em sistemas gráficos, utilizam-se funções de interpolação não lineares, como as curvas de aceleração senoidais ou equações cúbicas de suavização.

SLOW OUT

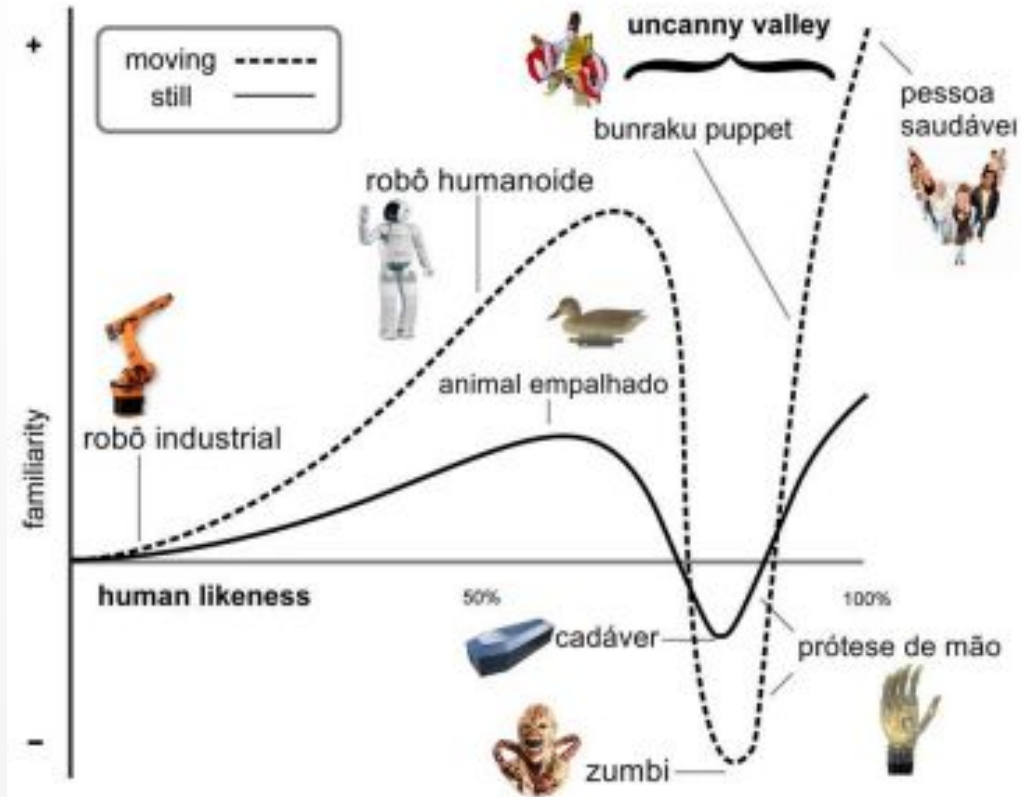
SLOW IN



O Fenômeno do Vale da Estranheza (Uncanny Valley)

Ao projetar animações que mimetizam seres vivos ou robôs humanoides, os desenvolvedores enfrentam o desafio psicológico do Vale da Estranheza (Uncanny Valley). Este conceito demonstra que a familiaridade e a aceitação humana aumentam à medida que o objeto se torna mais parecido com o real, mas despencam abruptamente em repulsa quando a semelhança se aproxima da perfeição sem atingi-la totalmente.

Movimentos estáticos ou pequenas falhas de interpolação em modelos altamente realistas fazem o objeto ser percebido como um zumbi ou um cadáver. Portanto, na computação gráfica, a fidelidade da animação e o refinamento do movimento devem evoluir em estrito equilíbrio com o realismo dos modelos tridimensionais e texturas.



Uncanny Valley

Bonitinho



Estranho



Realista



Paradigmas de Animação: Ativa, Passiva e Baseada em Dados

O desenvolvimento de sistemas gráficos trabalha com três grandes paradigmas de controle de movimento: animação ativa, passiva e baseada em dados (data-driven).

A animação ativa baseia-se na definição manual de quadros-chave (keyframes) e curvas de interpolação projetadas por artistas ou programadores. A animação baseada em dados utiliza o mapeamento direto de arquivos de captura de movimento real para transpor a performance humana para esqueletos virtuais.

Por fim, a animação passiva apoia-se em simulações computacionais onde o movimento é gerado automaticamente pela aplicação de leis físicas na CPU ou GPU, eliminando a necessidade de animações pré-gravadas.

Fundamentos da Simulação Física em Computação Gráfica

A simulação física substitui as aproximações visuais pela aplicação rigorosa das equações clássicas da mecânica newtoniana sobre as primitivas gráficas. Parâmetros cinemáticos e dinâmicos fundamentais, tais como aceleração, velocidade instantânea, atrito superficial e constantes elásticas de molas, passam a governar a matriz de transformação do objeto.

A cada iteração do laço principal do OpenGL, a aplicação calcula as forças resultantes atuando sobre o corpo e resolve numericamente as equações de movimento. Esse paradigma confere alto grau de realismo e imprevisibilidade ao ambiente virtual, permitindo que os objetos reajam dinamicamente a perturbações externas.

Representação de Orientação Espacial: Ângulos de Euler

Para animar a rotação de corpos rígidos no espaço tridimensional, é comum utilizar o sistema de coordenadas composto pelos Ângulos de Euler, que segmentam a orientação em três eixos independentes: guinada (yaw), arfagem (pitch) e rolamento (roll). **A guinada controla o movimento em torno do eixo vertical, a arfagem gerencia a inclinação lateral e o rolamento coordena a rotação sobre o próprio eixo longitudinal.**

Embora intuitiva e simples de implementar por meio de matrizes consecutivas de rotação no OpenGL, essa metodologia apresenta limitações matemáticas severas. O cálculo sequencial desses ângulos pode fazer o sistema perder um grau de liberdade espacial, fenômeno conhecido como travamento de cardã (gimbal lock).

Estudo de Caso Clássico: O Modelo de Asteroids

O clássico jogo Asteroids exemplifica de forma elementar a aplicação integrada de movimento vetorial, rotação e controle de projéteis em computação gráfica bidimensional. O sistema computacional deve gerenciar de forma independente os vetores de velocidade e posição dos asteroides em rotação contínua.

Simultaneamente, a nave do jogador atualiza sua orientação angular com base nos comandos do teclado, aplicando uma força de empuxo que modifica sua aceleração linear. Os tiros disparados herdam a posição inicial da nave e adicionam um vetor de velocidade constante, exigindo que o motor gráfico atualize e verifique todos esses elementos dinamicamente a cada quadro.

Sistemas de Colisão e Interação: Contato e Forças

Os sistemas de colisão e interação são projetados para computar o contato físico e a influência mútua entre os objetos, geometrias e partículas que compõem o cenário virtual. A interação se manifesta por meio do cálculo e aplicação de forças físicas atuantes que alteram o estado de movimento dos corpos envolvidos.

Entre as forças mais comuns simuladas em tempo real destacam-se a gravidade constante, o atrito desacelerador com o ar ou superfícies e campos magnéticos direcionais. Para que essas forças influenciem o ambiente de forma correta, o motor gráfico deve executar duas etapas sequenciais: a detecção do contato e a resposta à colisão.

O Pipeline de Detecção de Colisão

A detecção de colisões consiste no processo algorítmico de identificar se dois ou mais objetos geométricos compartilham o mesmo espaço físico no ambiente virtual.

Trata-se de um problema computacional complexo devido ao alto custo de verificar a interseção de dados geométricos refinados em tempo real. Para otimizar o processamento, o pipeline de detecção é dividido em duas fases distintas: a fase ampla (broad phase) e a fase estreita (narrow phase).

A fase ampla elimina rapidamente os objetos distantes usando testes matemáticos simples, enquanto a fase estreita executa checagens detalhadas e precisas apenas nos corpos com real potencial de colisão.

Detecção Rígida: Testes Vértice a Vértice

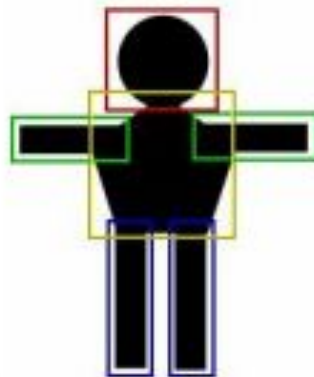
O método de detecção de colisão vértice a vértice representa a abordagem mais detalhada e precisa para avaliar a interseção entre malhas poligonais complexas. Neste modelo, o algoritmo checa se os vértices de um polígono ultrapassaram os limites das faces ou arestas de outra geometria tridimensional.

Contudo, o custo computacional dessa abordagem escala de forma quadrática em relação ao número total de polígonos presentes na cena. Devido a essa complexidade algorítmica, o teste puramente geométrico vértice a vértice torna-se inviável se aplicado indiscriminadamente em tempo real, demandando o uso estratégico de geometrias simplificadas envolventes.

Otimização por Geometria Simplificada: Volumes Envolventes

Para viabilizar a execução estável de taxas de quadros elevadas, a computação gráfica adota volumes envolventes (bounding volumes) para simplificar as checagens geométricas. Essa técnica consiste em encapsular a malha poligonal detalhada de um objeto complexo dentro de uma forma geométrica primitiva muito mais simples, como caixas ou esferas.

Os algoritmos matemáticos de detecção realizam os testes primários diretamente sobre esses volumes simplificados, que possuem equações de interseção triviais e rápidas. Caso os volumes envolventes não colidam, a geometria interna complexa é completamente ignorada, economizando tempo valioso de processamento da CPU.



Caixas Envolventes Alinhadas aos Eixos (AABB)

O modelo Axis-Aligned Bounding Box (AABB) delimita o objeto por meio de uma caixa cujas faces são mantidas paralelas aos eixos ortogonais coordenados X, Y e Z do sistema gráfico. A grande vantagem da técnica AABB reside na simplicidade de seu teste matemático, que requer apenas comparações diretas entre as coordenadas mínimas e máximas das caixas envolventes.

Se o valor máximo de coordenada de um objeto for menor que o valor mínimo de outro em qualquer um dos eixos, a colisão é descartada imediatamente. A limitação surge quando o objeto interno rotaciona, exigindo o recálculo dinâmico das dimensões da caixa envolvente para evitar falsos positivos.

Caixas Envolventes Orientadas (OBB) e Volumes Esféricos

Para resolver as falhas de precisão causadas pela rotação de objetos, utiliza-se a abordagem Oriented Bounding Box (OBB), na qual a caixa envolvente rotaciona solidariamente com a geometria interna. O teste de colisão em modelos OBB baseia-se no Teorema do Eixo Separador (SAT), exigindo projeções vetoriais mais complexas que elevam o custo computacional.

Outra alternativa comum são as esferas envolventes (bounding spheres), que permanecem totalmente invariantes à rotação. O teste de colisão entre duas esferas requer apenas calcular se a distância euclidiana entre os seus centros é menor do que a soma de seus respectivos raios, eliminando cálculos trigonométricos dispendiosos.

Resolução de Colisões e Conservação de Momento

Uma vez detectada a colisão pelas formas envolventes, o sistema deve aplicar o algoritmo de resposta para impedir a penetração mútua dos corpos e calcular as novas velocidades pós-impacto. A resposta física simula o impulso mecânico, aplicando leis de conservação do momento linear e angular e considerando o coeficiente de restituição dos materiais.

Corpos perfeitamente elásticos rebatem conservando a energia cinética, enquanto corpos inelásticos absorvem o impacto e reduzem a velocidade relativa. Programar esse comportamento envolve reposicionar os objetos para fora da zona de interseção e atualizar seus vetores de velocidade instantaneamente no laço lógico da aplicação.

Erros Comuns: O Fenômeno de Tunelamento (Tunnelling)

Um dos equívocos mais frequentes no desenvolvimento de simulações físicas e jogos é o fenômeno de tunelamento, também conhecido como transpassamento de barreiras.

Esse erro ocorre quando um objeto desloca-se a uma velocidade tão elevada, ou a taxa de atualização lógica é tão baixa, que o corpo salta de uma posição para outra atravessando completamente uma parede sem disparar o gatilho de colisão.

Como a detecção padrão avalia instantes discretos de tempo, o contato ocorrido no espaço intermediário é ignorado pelo algoritmo. Resolver o tunelamento exige implementar algoritmos de detecção contínua de colisão (CCD), que varrem o volume gerado pela trajetória do corpo ao longo do tempo.

Tendências Modernas: Simulação Física Acelerada por GPU

Com o advento das GPUs modernas e das APIs gráficas de baixo nível, a tendência atual concentra-se na migração completa dos sistemas de animação e colisão para a placa de vídeo. Utilizando recursos avançados como os Compute Shaders do OpenGL moderno, torna-se viável processar milhões de partículas ou colisões de corpos rígidos de forma massivamente paralela.

Essa arquitetura elimina o gargalo histórico de transferência de dados entre a memória RAM da CPU e a VRAM da GPU. Consequentemente, sistemas contemporâneos conseguem renderizar interações físicas de altíssima complexidade e fluidos dinâmicos em tempo real com taxas de quadros estáveis.

A Integração do Sistema Gráfico

A construção de uma aplicação interativa tridimensional bem-sucedida requer o alinhamento harmônico entre as técnicas estéticas de animação e o rigor físico dos modelos de colisão.

O pipeline programável do OpenGL oferece a infraestrutura necessária para processar transformações geométricas complexas e deformações baseadas em princípios clássicos com alta eficiência.

Paralelamente, o uso inteligente de estruturas de dados otimizadas e volumes envolventes mitiga o custo computacional inerente aos testes contínuos de interseção. Dominar a matemática de vetores, matrizes e a física clássica consolida-se, portanto, como o alicerce técnico essencial para qualquer engenheiro de software gráfico.

Comandos para Animação (Controle de Tempo e Loop)

A animação em tempo real depende diretamente de medir o tempo decorrido entre os quadros para que a velocidade do movimento seja independente da taxa de atualização da CPU/GPU.

GLFW.GET_TIME(): Retorna o tempo decorrido (em segundos) desde que a biblioteca GLFW foi inicializada.

GLFW.SWAP_BUFFERS(WINDOW): Inverte os buffers de renderização (Double Buffering), exibindo a imagem que acabou de ser desenhada e liberando o buffer oculto para o próximo quadro.

Exemplo de Estrutura de Loop de Animação:

```
tempo_anterior = glfw.get_time()

while not glfw.window_should_close(window):
    tempo_atual = glfw.get_time()
    delta_time = tempo_atual - tempo_anterior # Tempo que o quadro
demorou
    tempo_anterior = tempo_atual

    # Exemplo: Translação baseada em física (Posição = Velocidade *
Tempo)
    posicao_x += velocidade_x * delta_time

    # Renderiza e inverte a tela
    glfw.swap_buffers(window)
    glfw.poll_events()
```

Comandos para Interação (Captura de Entradas)

A interação ocorre quando o usuário altera o estado do sistema. O GLFW oferece duas abordagens: verificação imediata (polling) ou funções de retorno (callbacks).

Abordagem por Polling (Verificação Imediata)

Ideal para movimentos contínuos (ex: segurar uma tecla para mover a nave).

`glfw.get_key(window, KEY_NAME)`: Verifica o estado atual de uma tecla específica no exato momento da chamada (PRESS ou RELEASE).

Comandos para Interação (Captura de Entradas)

Abordagem por Callbacks (Eventos Discretos)

Ideal para ações pontuais que não podem se repetir acidentalmente (ex: pausar o jogo, disparar um tiro, mudar o tipo de colisão).

`glfw.set_key_callback(window, callback_function)`: Registra uma função que será disparada automaticamente apenas quando uma tecla mudar de estado.

`glfw.set_cursor_pos_callback(window, callback_function)`: Captura as coordenadas (**X**, **Y**) do mouse na tela a cada pixel movimentado, essencial para rotacionar câmeras ou objetos 3D.

Questionário



Exercícios

1. Utilizando a lógica de colisão (AABB ou esférica) já existente em seu código, faça com que o objeto mude de cor imediatamente ao detectar um impacto. Use o comando `glUniformMatrix4fv` para atualizar a posição de recuo e renderize o objeto na cor vermelha enquanto a colisão persistir.
2. Crie uma tecla de atalho no GLFW (como a tecla 'B') que ative um "modo de depuração". Quando pressionada, o OpenGL deve alternar a renderização das caixas de colisão para o modo de linhas (`glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)`) para que os volumes envolventes fiquem visíveis sobre os objetos.

Exemplo

```
import glfw

# 1. Variáveis de Animação e Física
pos_jogador = [0.0, 0.0]
vel_jogador = 2.0 # Unidades por segundo
tamanho = 0.5

# Caixa Envolvente da Barreira Fixa (AABB)
barreira_min = [1.0, 1.0]
barreira_max = [1.5, 1.5]

# Loop Principal (Animação)
while not glfw.window_should_close(window):
    # Calcula Delta Time
    dt = calcular_delta_time()
```